



AFTER-SCHOOL PROGRAM 2025

# The Hub for Young Next-gen Knowledge

**T · H · Y · N · K**

THE · HUB · FOR · YOUNG · NEXT-GEN · KNOWLEDGE

Where Ancient Indian Wisdom meets Modern Programming  
— a unique after-school program blending Vedic  
Mathematics with Scratch, HTML, CSS & JavaScript.



Vedic Maths



Coding

Grades 4 – 7

16 Weeks • Once a Week

School Computer Teachers

WHAT T.H.Y.N.K STANDS FOR

**T** The**H** Hub for**Y** Young**N** Next-gen**K** Knowledge**16****2****45min****0**

| Program Duration                      |          | Ready-to-Teach Sessions         |            | Vedic Sutras Taught           |                        |
|---------------------------------------|----------|---------------------------------|------------|-------------------------------|------------------------|
| Once per week, 45 min                 |          | Two tracks × 16 weeks           |            | Each mapped to a code concept |                        |
| T.H.Y.N.K                             | Why This | The Connection                  | Grades 4–5 | Grades 6–7                    | Materials Requirements |
|                                       |          |                                 |            |                               |                        |
| Phases per Track                      |          | Extra Hardware Cost             |            |                               |                        |
| Foundation → Build → Deepen → Present |          | Uses your existing computer lab |            |                               |                        |

THE OPPORTUNITY

Why Vedic Maths + Coding?

Both Vedic Mathematics and Computer Programming share the same DNA — they are systems of precise, pattern-based rules applied to solve problems efficiently. Teaching them together creates a multiplier effect on a child's logical reasoning.



IN

## Same Cognitive Skills

Vedic sutras train the brain to look for patterns, shortcuts, and elegant solutions — exactly what a programmer does. Children who learn Vedic Maths first pick up coding **significantly faster** because they're already thinking algorithmically.

## Rooted in Indian Heritage

Vedic Mathematics comes from the *Atharva Veda*. Taught alongside modern coding, it gives students a pride of identity — showing that India's ancient scholars were the world's first programmers, writing algorithms in Sanskrit.



## Faster Mental Computation

Kids who practice Vedic techniques compute 3–5× faster mentally. In coding, this translates to faster debugging, quicker mental simulation of code logic, and the ability to estimate results before running a program.



## Sutras ARE Algorithms

The word *sutra* means "thread" — a rule you follow step by step. That's exactly what an algorithm is. Each Vedic *sutra* is a real algorithm; translating them into code makes both maths and coding instantly more concrete and understandable.



## Perfect for Schools

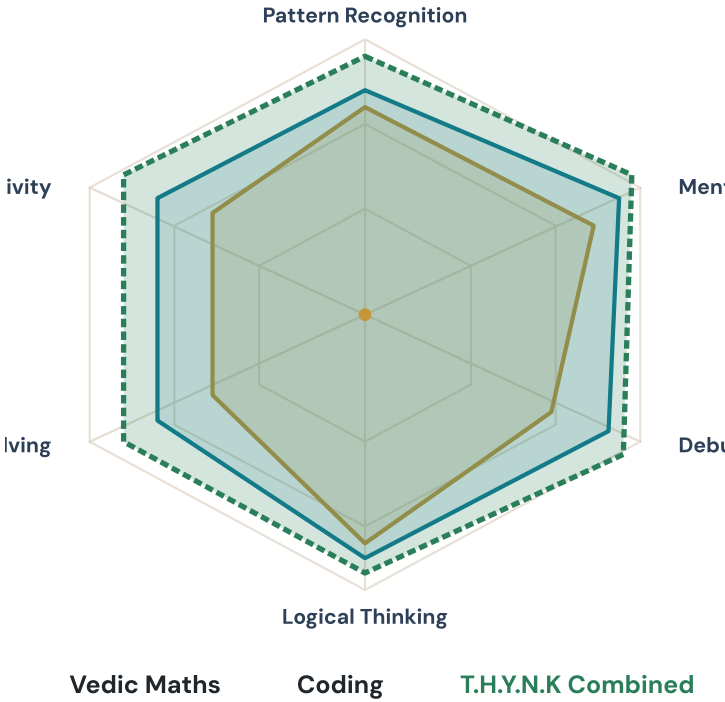
CBSE and ICSE schools already cover basic Vedic methods in Grade 6–8 syllabi. This program builds on what teachers already know, requires no new equipment, and aligns perfectly with NEP 2020's emphasis on coding and critical thinking.



## Teacher-Friendly Design

Every session comes with a complete teacher guide, student worksheets, digital resources, and step-by-step instruction scripts. A computer teacher with no Vedic Maths background can confidently deliver

Week 1 after a single half-day training session.



COGNITIVE IMPACT

# Six Skills, Doubled in Strength

When Vedic Maths and Coding are taught together, each skill reinforces the other — creating children who are significantly stronger in all six areas than those who learn either subject in isolation.

|                          |               |
|--------------------------|---------------|
| Pattern Recognition      | Combined +47% |
| Mental Computation Speed | Vedic: 3–5×   |
| Logical Sequencing       | Code-driven   |

THE DIRECT LINK

# Every Sutra Has a Code Twin

This is not analogy — these are exact, one-to-one mappings. The same mental operation that a student uses in the Vedic technique is the same operation their program performs.

|  VEDIC SUTRA |  SCRATCH (GR 4-5) |
|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <div>Ekadhikena Purvena</div> <div>+1 to previous</div>                                         | <div>repeat / change by 1</div> <div>Loop block</div>                                                 |
| <div>Nikhilam</div> <div>Complement from base</div>                                             | <div>set [variable] to</div> <div>Variable + operator</div>                                           |
| <div>Urdhva Tiryak</div> <div>Vertically &amp; crosswise</div>                                  | <div>repeat inside repeat</div> <div>Nested loops</div>                                               |
| <div>Gunita Samuchaya</div> <div>Verify by digital root</div>                                   | <div>if answer = correct</div> <div>Conditional block</div>                                           |
| <div>Anurupyena</div> <div>Proportional scaling</div>                                           | <div>set size to % of score</div> <div>Proportional looks</div>                                       |

**Shunyam**

Special-case = zero

**if [ ] = 0 then**

Zero condition

**Paravartya**

Transpose &amp; adjust

**set x / set y blocks**

Coordinate swap

 **HTML/JS (GR 6–7)****for loop / while loop**

JS iteration

**let a = base - n**

JS variables

**for(i) { for(j) { } }**

Nested JS loops

**digitalRoot() % 9**

Modulo operator

**width: 50% in CSS**

CSS proportions

**if (x === 0) { }**

JS conditionals

**function swap(a,b)**

JS functions

**THE BRIDGE**

## 5 Ways Vedic Maths Directly Powers Coding

These are not metaphors — these are direct, concrete mappings that we use in every lesson. Each Vedic sutra becomes a coding concept children can see, build,

and run.

## 01

### Ekadhikena Purvena

*"By one more than the previous one"*

The simplest Vedic sutra — add 1 to what came before — is literally a *counter loop*. Children who understand this sutra immediately grasp why computers count the way they do.

#### VEDIC MATHS

Multiply  $1/19 = 0.052631...$

Each digit: previous digit  $\times 2$

**Pattern:**  $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow \dots$

Squaring:  $25^2 = (2 \times 3) \mid 25 = \mathbf{625}$

$35^2 = (3 \times 4) \mid 25 = \mathbf{1225}$

$45^2 = (4 \times 5) \mid 25 = \mathbf{2025}$

#### IN SCRATCH / JS

This IS a loop!

```
-- Scratch --
repeat 10
  change score by 1
end

// JavaScript
for (let i = 1; i <= 10; i++) {
  console.log(i);
}
```

## 02

### Nikhilam Navatashcaramam Dashatah

*"All from 9, the last from 10"*

This sutra computes complements from the nearest base (10, 100, 1000). In programming, this is identical to the concept of *variables* that store intermediate values and the *subtraction operator*.

#### VEDIC MATHS

$$97 \times 96 = ?$$

Base = 100

Deficit of 97 = -3

Deficit of 96 = -4

Left:  $97 - 4 = 93$  (or  $96 - 3$ )

Right:  $(-3) \times (-4) = 12$

**Answer: 9312**

#### IN SCRATCH / JS

Variables store the "deficits"

```
let a = 97, b = 96;
let base = 100;
let defA = base - a; // 3
let defB = base - b; // 4
let left = a - defB; // 93
let right = defA * defB; // 12
let ans = left*100 + right; // 9312
```

## 03

### Urdhva Tiryakbhyam

*"Vertically and Crosswise"*

The Vedic method for multi-digit multiplication — cross-multiply and carry. This maps directly to *nested loops*, where an inner loop runs inside an outer loop, processing every combination of digits.

#### VEDIC MATHS

$$23 \times 41 = ?$$

Step 1 (vertical right):  $3 \times 1 = 3$

Step 2 (cross):  $2 \times 1 + 3 \times 4 = 14$ , write 4, carry 1

Step 3 (vertical left):  $2 \times 4 = 8 + \text{carry } 1 = 9$

**Answer: 943**

 **IN JS — NESTED LOOPS**

Outer loop = tens, inner = units

```
// Multiplication table
for (let i = 1; i <= 10; i++) {
  for (let j = 1; j <= 10; j++) {
    console.log(i + "x" + j
      + "=" + (i*j));
  }
}
```

## O4

### Gunita Samuchaya

*"The product of the sum equals the sum of the product"*

The Vedic technique of *casting out 9s* — computing a digital root to verify calculations. In coding, this is identical to *testing and debugging*: running a quick check to verify your program's output is correct.

 **VEDIC VERIFICATION**

Check:  $47 \times 83 = 3901$ ?

Digital root of 47 =  $4+7 = 11 \rightarrow 2$

Digital root of 83 =  $8+3 = 11 \rightarrow 2$

Product of roots:  $2 \times 2 = 4$

Digital root of 3901 =  $3+9+0+1 = 13 \rightarrow 4 \checkmark$

*Answer is verified correct!*

 **IN JS — MODULO (%) OPERATOR**

Digital root uses mod 9!

```
function digitalRoot(n) {
  return n === 0 ? 0
    : 1 + (n - 1) % 9;
}

// Verify multiplication
let a = 47, b = 83;
let result = a * b; // 3901
console.log(digitalRoot(a)
  * digitalRoot(b) % 9)
```

```

    === digitalRoot(result) % 9);
    // true ✓

```

## 05 Anurupyena

"Proportionality"

The Vedic principle of scaling — if you know  $1\times$ , you know  $2\times$ ,  $3\times$ , any multiple. In coding, this maps directly to *functions*: write the rule once, call it with any input, and the proportional result scales automatically.

### VEDIC MATHS

Know  $11\times 7 = 77$ ?

Then instantly:  $110\times 7 = 770$

$1100\times 7 = 7700$

$0.11\times 7 = 0.77$

**Proportional scaling — same rule, different input.**

Also: CSS percentage layouts!

1 col = 100%, 2 col = 50%, 3 col = 33.33%

### IN JS — FUNCTIONS / CSS

Write once, scale infinitely

```

// One function, any scale
function multiply(base, factor) {
    return base * factor;
}

multiply(11, 7);    // 77
multiply(110, 7);   // 770

/* CSS proportionality */
.col-3 { width: 33.33%; }
.col-2 { width: 50%; }
.col-1 { width: 100%; }

```

## PROGRAM STRUCTURE

# Two Tracks, One Philosophy

T.H.Y.N.K runs as two parallel tracks matched to grade bands. Both use the same pedagogical approach: introduce a Vedic sutra, find its coding twin, then build a project that makes both concepts visible and fun.

## TRACK 1

## Vedic Maths + Scratch

Visual block coding meets ancient arithmetic — perfect first steps into the world of programming.

## FOR

**Grades 4 & 5**

## DURATION

**16 Weeks**

## SESSION LENGTH

**45 minutes**

## TOOLS USED

**Scratch 3.0 (free)**

## VEDIC FOCUS

**7 Sutras**

## FINAL PROJECT

**Vedic Maths Game**

- Ekadhikena Purvena → Loops & Patterns in Scratch
- Nikhilam → Variables & Subtraction
- Multiplication tricks → Calculator projects
- Digital roots → Conditionals (if/else)
- Proportions → Sprite sizing & animations
- Verification → Testing & debugging
- Mini-project: Vedic Maths Quiz Game

→ Final project: Student-designed Vedic game

## TRACK 2

# Vedic Maths + Web Code

Real-world HTML, CSS and JavaScript powered by the mental shortcuts of Vedic arithmetic.

### FOR

**Grades 6 & 7**

### DURATION

**16 Weeks**

### SESSION LENGTH

**45 minutes**

### TOOLS USED

**Browser + Notepad / VS Code**

### VEDIC FOCUS

**8 Sutras**

### FINAL PROJECT

**Vedic Maths Website**

- HTML structure → Sutra categories & rules
- CSS proportions → Anurupyena scaling
- JS variables → Unknown quantities in sutras
- JS operators → All 4 Vedic operations
- Conditionals → Shunyam special-case rules
- Functions → Reusable sutras
- Loops → Urdhva Tiryakbhyam steps
- Final project: Interactive Vedic Maths teach-back website

## THE JOURNEY

# 16-Week Roadmap

Both tracks follow the same 4-phase arc — from first principles to a fully working final project. Every phase ends with a milestone the student can show to parents.

## TRACK 1 — GRADES 4 & 5 (SCRATCH)

1-4

WEEKS 1-4

### Phase 1: Foundations

Meet Vedic Sutras and Scratch. Make sprites move, talk, and count. First calculator built using the  $\times 11$  trick.

Ekadhikena   Motion Blocks   Variables

5-8

WEEKS 5-8

### Phase 2: Building Tools

Build calculators, health-bar games, and a full quiz using Nikhilam and squaring tricks. Mini-project at Week 8.

Nikhilam   Squaring  $\times 5$    Quiz Game 🏆

9-12

WEEKS 9-12

### Phase 3: Deeper Thinking

Digital roots  $\rightarrow$  if/else logic. Proportions  $\rightarrow$  sprite scaling. Nested loops  $\rightarrow$  yantra grid art. Debugging mindset unlocked.

Conditionals   Nested Loops   Anurupyena

13-16

WEEKS 13-16

### Phase 4: Final Project 🎉

Design, build, test and present a complete Vedic Maths game. Demo Day in Week 16 with parents and school staff.

Own Game   Certificate   Demo Day

## TRACK 2 — GRADES 6 & 7 (HTML/CSS/JS)

1-4

WEEKS 1-4

### Phase 1: Structure & Style

HTML as a "sutra system" for browsers. CSS box model = Vedic geometry. Build a beautifully styled Sutras reference page.

HTML Tags   CSS Selectors   Box Model

5–8

WEEKS 5–8

Phase 2: Making It Work

JavaScript variables as ancient "yavat-tavat." All 4 Vedic operations in code. Mini-project: full interactive calculator webpage.

JS Variables   Operators   Calculator 🏆

9–12

WEEKS 9–12

Phase 3: Advanced Logic

Loops visualise Urdhva Tiryak steps. Each sutra becomes a JS function. Arrays hold digit sequences. Animated Vedic yantra built.

Functions   Loops   CSS Animation

13–16

WEEKS 13–16

Phase 4: Teaching Website 🇮🇳

4–page website that teaches 2–3 Vedic sutras interactively. Live demo and code walkthrough on Demo Day.

Own Website   Certificate   Demo Day



|                     |       |
|---------------------|-------|
| Vedic Warm-Up       | 8 min |
| Sutra → Code Bridge | 5 min |

**Build / Code Time**

25 min

**Share & Reflect**

7 min

**SESSION DESIGN**

## Every 45 Minutes Is Structured the Same Way

The session structure is intentional. Students always start with mental Vedic maths (no screens, builds confidence), then bridge to code, then build, then reflect. This rhythm becomes second nature by Week 3.

**Vedic Warm-Up — 8 min**

5 mental maths problems on today's sutra. No calculators. Students compete to solve fastest — builds the mental speed that pays off in debugging.

**Sutra → Code Bridge — 5 min**

Teacher draws the explicit connection: "This sutra's Step 2 is exactly what this block / line of code does." Students write it on their worksheet.

**Build / Code Time — 25 min**

Hands on keyboard. Guided worksheet + teacher circulation. Pairs encouraged. Fast finishers get extension challenges. Teacher never gives answers — only asks questions.

**Share & Reflect — 7 min**

Two students show their screen. Class spots one thing that works well and one thing to improve. Everyone writes: "Today I connected \_\_ to \_\_." Homework: teach the sutra to one family member.

**FULL CURRICULUM**

## Grades 4–5: Vedic Maths + Scratch

16 weeks, once a week, 45 minutes per session. Each week pairs a Vedic technique with a Scratch concept — then students immediately build something with it.

**Scratch Track**

## Phase 1: Foundations

WEEKS 1–4

### WEEK 01

#### Welcome to T.H.Y.N.K! What are Vedic Maths & Scratch?

##### VEDIC

Introduction to the 16 Sutras. What is a sutra? (A rule!) Examples: multiplying by 11, squaring numbers ending in 5. Show how ancient Indians computed without calculators.

##### SCRATCH

Open Scratch. Meet Sprites, the Stage, and Blocks. Run a "Hello World!" — make the cat say "Namaste!" and wave. Understand what a program is.

 *Outcome: Students can explain what a sutra and a program have in common — both are step-by-step rules.*

### WEEK 02

#### Ekadhikena Purvena — Counting & Motion Blocks

##### VEDIC

Sutra: "By one more than the previous one." Practice:  $1/19$ ,  $1/29$  using this pattern. Spot doubling sequences: 1, 2, 4, 8, 16... What comes next?

##### SCRATCH

Motion blocks: move 10 steps, turn 15 degrees. Make sprite walk by pressing arrow keys. Each key press = "one more step." Worksheet: match the sutra step to the code step.

 *Outcome: Students see that "add 1 more" is the same action in Vedic counting and in making a sprite move. Mini-project: Walking cat that counts steps.*

### WEEK 03

#### Number Patterns → The Repeat Loop

##### VEDIC

Explore repeating patterns in multiplication tables. 9's table: 9, 18, 27, 36... digit sum always = 9! Eleven's table: digit mirror pattern 11, 22, 33... Spot the pattern, predict the next.

##### SCRATCH

The **repeat** block — let the computer repeat instead of you. Repeat 10: move 20, turn  $36^\circ$  → draws a perfect circle! Connection: the multiplication pattern IS the loop.

 *Outcome: Students build a spiral/polygon using repeat loops. They explain why  $360 \div 10 = 36^\circ$  using Vedic proportional thinking.*

### WEEK 04

#### Multiplying by 11 → Variables & Answers

##### VEDIC

##### SCRATCH

**Trick:**  $23 \times 11 = 2\_3$ , middle =  $2+3 = 5 \rightarrow 253$ .  $45 \times 11 = 4\_5$ ,  $4+5=9 \rightarrow 495$ . What if sum  $> 9$ ?  $78 \times 11$ :  $7+8=15$ , carry 1  $\rightarrow 858$ . Practice 10 examples mentally.

Sensing blocks: **ask** and **answer**. Variables: Number1, Result. Build a  $\times 11$  calculator: ask a 2-digit number, compute it using the Vedic trick as separate variable steps, say the answer.

 *Outcome: Students build their first calculator in Scratch. They see how the Vedic steps (split digits  $\rightarrow$  add middle  $\rightarrow$  join) become variable operations in code.*

## Phase 2: Building Tools

WEEKS 5–8

### WEEK 05

#### All from 9, Last from 10 $\rightarrow$ Subtraction & Score Bars

##### VEDIC

Nikhilam sutra:  $1000 - 386 = ?$  Take each digit from 9, last from 10: 6,1,4  $\rightarrow 614$ . Instant! Practice:  $10000 - 2847$ ,  $100 - 73$ . This is complement arithmetic — same as computers use!

##### SCRATCH

Build a **health bar game**. Variable: Lives = 3. Each time a ball touches the sprite: change Lives by  $-1$ . If Lives = 0: game over. This IS "all from [max], last from [max]" in code.

 *Outcome: Students build a working game with a live health counter. They explain the connection between the Nikhilam sutra and the Lives variable.*

### WEEK 06

#### Squaring Numbers Ending in 5 $\rightarrow$ Formulas & Functions

##### VEDIC

Any number ending in 5:  $n \times (n+1) \mid 25$ .  $35^2 = 3 \times 4 \mid 25 = 1225$ .  $75^2 = 7 \times 8 \mid 25 = 5625$ .  $105^2 = 10 \times 11 \mid 25 = 11025$ . No multiplication needed — pure pattern recognition.

##### SCRATCH

Advanced calculator: Ask for a number ending in 5. Compute using the Vedic formula (store tens digit in variable, multiply by tens+1, join 25). Display result. Compare with direct multiply to verify!

 *Outcome: Students build a Vedic squaring calculator and experience the joy of getting the same result two different ways — intuitive Vedic vs brute-force multiplication.*

### WEEK 07

#### Near-10 Multiplication $\rightarrow$ Scoring & Operators

##### VEDIC

Base 10 multiplication:  $8 \times 7 = ?$  Both near 10. 8 is  $-2$  from 10, 7 is  $-3$ . Cross:  $8 - 3 = 5$  (or  $7 - 2 = 5$ ). Product of deficits:  $2 \times 3 = 6$ . Answer: **56**. Try  $9 \times 6$ ,  $7 \times 6$ ,  $8 \times 9$  mentally.

##### SCRATCH

Scoring system: Green flag starts score=0. Each correct answer: score +10. Each wrong answer: score  $-5$ . Use **operator blocks (+, -,  $\times$ )** directly.

The deficits in Vedic = the penalty/bonus in the game!

 *Outcome: Students build a working quiz with live score tracking using Vedic multiplication as both the subject and the scoring logic.*

#### WEEK 08 — MINI PROJECT

### Build a Vedic Maths Quiz Game in Scratch

#### VEDIC CONTENT

Quiz covers:  $\times 11$  trick, squaring ending in 5, near-10 multiplication, Nikhilam subtraction. 5 questions, timed (30 seconds each). Use all Vedic shortcuts learned so far.

#### SCRATCH FEATURES

Must include: Variables (Score, Timer, Lives), Ask/Answer blocks, If/Else for correct/wrong, Repeat loop for countdown, Say block for feedback. Full working game!

 *Students share and play each other's quiz games. Peer testing = informal code review! Teachers assess variables, loops, and conditionals.*

## Phase 3: Deeper Concepts

WEEKS 9–12

#### WEEK 09

### Digital Roots → Conditionals (If / Else)

#### VEDIC

Digital root = keep adding digits until single digit.  
 $9 \rightarrow 9$ ,  $10 \rightarrow 1$ ,  $11 \rightarrow 2$ ,  $18 \rightarrow 9$ ,  $19 \rightarrow 1$ . Rules: if digital root = 9  $\rightarrow$  divisible by 9. If = 3 or 6  $\rightarrow$  divisible by 3.  
 Instant divisibility check!

#### SCRATCH

If/Else blocks: **If** (digital root of answer = 9) **then** say "Divisible by 9!" **else** say "Not divisible by 9."  
 Build a divisibility checker — the Vedic rule IS the condition!

 *Outcome: Students build a divisibility tester. The if/else logic comes naturally because they already know the Vedic rule that goes inside the condition.*

#### WEEK 10

### Proportions (Doubling & Halving) → Sprite Sizing & Looks

#### VEDIC

Anurupyena: proportional scaling. If  $12 \times 5 = 60$ , then  $24 \times 5 = 120$ ,  $6 \times 5 = 30$ ,  $1.2 \times 5 = 6$ . Double the number, double the answer. Half the number, half the answer. Apply to areas:  $2 \times$  length =  $4 \times$  area.

#### SCRATCH

Looks blocks: set size to %, change size by. Make sprite grow when score increases (size = score  $\times$  2), shrink when score drops. Show proportional relationship visually. Also: set costume effects proportionally.

 *Outcome: Students create an animated "score creature" that visually scales proportionally with the game score — direct visual proof of Anurupyena.*

### WEEK 11

#### Vertically & Crosswise → Nested Loops & Grid Art

##### VEDIC

Urdhva Tiryakbhyam: 2-digit × 2-digit multiplication in 3 steps — right column, cross, left column. Process every position systematically. This is exactly what a nested loop does!

##### SCRATCH

Nested repeat loops: outer loop rows (move down 20), inner loop columns (stamp, move right 20). Creates a grid — exactly like filling in a multiplication table position by position. Draw Vedic yantra art!

 *Outcome: Students produce a stamp-art grid resembling a Vedic magic square. They narrate: "The outer loop is the tens digit, the inner loop is the units digit."*

### WEEK 12

#### Gunita Samuchaya — Testing & Debugging Like a Vedic Scholar

##### VEDIC

Verification sutra: use digital roots to check any multiplication.  $47 \times 83 = 3901$ ?  $\text{Root}(47) = 2$ ,  $\text{Root}(83) = 2$ ,  $2 \times 2 = 4$ .  $\text{Root}(3901) = 4$ . ✓ If they don't match → ERROR. This is a built-in error-detection system!

##### SCRATCH

Debugging session: teacher provides 3 broken Scratch projects. Students find bugs using: check variables panel, add say blocks to test values, trace execution step by step. Debugging = Vedic verification mindset.

 *Outcome: Students fix all 3 buggy projects. Key insight: "Checking is part of the work, not extra work" — same in Vedic maths and in programming.*

## Phase 4: Final Project

WEEKS 13–16

### WEEK 13

#### Project Planning — Design Your Vedic Game

##### VEDIC

Students choose which Vedic sutra their game will teach. Complete a "Game Design Sheet": what sutra, what question format, how score works, what happens when correct/wrong, what's the win condition.

##### SCRATCH

Sketch the Scratch layout: which sprites, which variables, which events. Draw a flowchart: Green Flag → Ask → Check → Score → Win/Lose. Map every box to the Scratch block that will do it.

 *Deliverable: Completed Game Design Sheet with Vedic sutra chosen, flowchart drawn, and sprites listed. Teacher approves before building starts.*

#### WEEK 14

### Project Build — Week 1

#### VEDIC

Each student must include at least 2 Vedic sutra questions. The Vedic calculations should be broken into steps matching the code (e.g., three separate variables for the three steps of  $\times 11$  trick).

#### SCRATCH

Build core game loop: ask question, get answer, check if correct, update score, go to next question. All variables named clearly. Teacher circulates and helps with debugging. Peer pair support encouraged.

 *By end of class: Core loop working (even with just 1 question). Students save and share project link.*

#### WEEK 15

### Project Polish — Test, Debug, Beautify

#### VEDIC

Students write a 5-question Vedic Maths test on paper using their chosen sutra. They swap papers with a classmate and solve using the sutra. This reveals if the sutra is correctly represented in their game.

#### SCRATCH

Add: start screen, end screen with "Play Again" button, sound effects, custom backdrop, at least 3 questions, timer, and high score. Use a classmate to test — if they're confused, fix the UX. Testing = verification.

 *Final game complete, saved, and shared. Each student records a 60-second voice note explaining their sutra and how it appears in their code.*

#### WEEK 16 — DEMO DAY

### T.H.Y.N.K Demo Day — Present & Celebrate!

#### PRESENTATION

Each student presents for 2 minutes: "My game teaches [sutra name]. This sutra means... In my game, you need to... The sutra helps because..." Judges (teachers, parents) ask: "How did Vedic maths help you code this?"

#### DEMO

Live demo of game on screen. Audience plays 1 round. Student explains one piece of code (open the blocks, point to it). Certificate of completion. Games published to class Scratch studio for all to play.

 *Students receive T.H.Y.N.K Certificate. Best projects featured in school newsletter. All games archived in school Scratch studio for future students.*

## FULL CURRICULUM

# Grades 6–7: Vedic Maths + HTML / CSS / JS

16 weeks, once a week, 45 minutes per session. Students build real websites while discovering that HTML, CSS and JavaScript are just Vedic sutras written in a different language.



## Phase 1: Structure & Style

WEEKS 1–4

## WEEK 01

### What is a Webpage? HTML = A Sutra for Browsers



What is a sutra? A rule with a name, a structure, and a predictable outcome. The Vedic texts contain 16 sutras and 13 sub-sutras — exactly like HTML has tags and attributes. Structure = sutra.



Write first HTML file in Notepad/VS Code. Tags: `<html>`, `<head>`, `<body>`, `<h1>`, `<p>`. Open in Chrome. Students add their name, school, and a Vedic sutra quote. Save, refresh, see changes instantly.

Outcome: Each student has a personal "about me" webpage live in Chrome — their first HTML file.

## WEEK 02

### HTML Lists & Tables — Organising Sutras



The 16 Vedic Sutras listed: Ekadhikena, Nikhilam, Urdhva Tiryak... each has a Sanskrit name, English meaning, and use case. Classification and hierarchy — just like HTML elements!



`<ul>`, `<ol>`, `<li>` for lists. `<table>`, `<tr>`, `<td>` for the multiplication table. Build an HTML page: "The 16 Vedic Sutras" — one table row per sutra with Sanskrit name, meaning, and example.

 *Outcome: Students create a well-structured HTML reference page of all 16 Vedic Sutras.*

### WEEK 03

## CSS Basics — Anurupyena (Proportions) in Style

### VEDIC

Anurupyena: proportional relationships. If font is 16px, a heading should be  $2 \times = 32\text{px}$ . Line height should be  $1.5 \times$  font size. Spacing: margin =  $1 \times$  font, padding =  $0.5 \times$  font. All proportions flow from one base unit — exactly like Vedic scaling.

### CSS

Link CSS file to HTML. Selectors: body, h1, p, table. Properties: color, background-color, font-size, font-family. Students style their Vedic Sutras page. Choose a colour scheme inspired by India: saffron, white, green.

 *Outcome: Sutras page is now beautifully styled. Students discover that good CSS proportions feel the same as Vedic proportional thinking.*

### WEEK 04

## CSS Box Model — Vedic Geometry & Squared Areas

### VEDIC

Vedic geometry (Sulbasutras — pre-dates Pythagoras!): area of square =  $\text{side}^2$ . The 3-4-5 right triangle was known. Box inside box inside box — exactly the CSS box model. Content → padding → border → margin.

### CSS

padding, margin, border, width, height. box-sizing: border-box. Build a "Vedic yantra" card layout using nested divs: outer border (margin), decorated frame (border), breathing room (padding), content in centre.

 *Outcome: Students create a styled Vedic Yantra card using only CSS box model properties. They narrate each layer using Vedic geometry language.*

## Phase 2: Making it Work with JavaScript

WEEKS 5–8

### WEEK 05

## Intro to JavaScript — Variables as Unknown Quantities

### VEDIC

In Vedic algebra (Bija Ganita), unknowns were called "yavat-tavat" (as much as). A variable is an unknown that takes a value. This is identical to algebra AND identical to JavaScript variables. Ancient India had variables!

### JS

script tags. let, const. prompt() to get input. alert() to show output. console.log() for testing. First JS program: ask two numbers, multiply them (using the  $\times 1$  trick for type conversion), show the result.

 *Outcome: Students write their first JavaScript and connect it directly to Vedic algebraic unknowns — variables were not invented in 1950s, they were used in 1000 BCE India.*

## WEEK 06

### JS Operators — All Four Vedic Operations

#### VEDIC

All four operations and their Vedic shortcuts: + (Sankalana — addition), - (Vyavakalana — subtraction), × (Gunana — near-base method), ÷ (Bhagana — Paravartya division). Show the shortcut method for each, then verify with JS.

#### JS

+, -, \*, /, % (modulo). Math.pow(), Math.sqrt(). Build a 4-operation Vedic calculator HTML page: input two numbers, four buttons (one per operation), result displayed in a div. CSS styled in India tricolour.

 *Outcome: Students have a working web calculator. They also know the Vedic Sanskrit name for each operation — connects cultural pride to technical skill.*

## WEEK 07

### JS Conditionals — Shunyam & Special Cases

#### VEDIC

Sutra: "Shunyam Saamyasamuccaye" — if the sum is the same on both sides, that quantity is zero. Special case detection! Vedic maths has rules that say "IF [condition] THEN [special shortcut] ELSE [general method]."

#### JS

if / else if / else. Comparison operators: ===, !==, >, <. Build divisibility tester: input a number, JS checks digital root and says "Divisible by 3/9/neither." The Shunyam condition becomes an if-statement.

 *Outcome: Students see if/else as the direct equivalent of Vedic "special case rules" — the code structure was already implicit in the sutra.*

## WEEK 08 — MINI PROJECT

### Build a Full Vedic Maths Calculator Webpage

#### VEDIC

Integrates: ×11 shortcut, squaring-ending-5, Nikhilam subtraction, near-10 multiplication, digital root check. Calculator has a "Show Vedic Steps" button that reveals the sutra's intermediate steps alongside the answer.

#### HTML + CSS + JS

HTML: input fields, buttons, result area, steps area. CSS: fully styled, responsive, uses flexbox for layout. JS: event listeners, if/else for choosing which sutra to apply based on the numbers entered. Three files, one cohesive product.

 *Deliverable: A hosted, working Vedic Maths calculator with step-by-step Vedic explanation panel. Teachers save these as portfolio pieces.*

## Phase 3: Loops, Functions & Advanced Ideas

WEEKS 9–12

### WEEK 09

#### JS Loops — Urdhva Tiryakbhyam Step by Step

##### VEDIC

Urdhva Tiryakbhyam for 2×2 digit multiplication — 3 passes: right column, crosswise, left column. Then for 3×3 digit — 5 passes. Each pass is a repeating pattern over digit positions. The loop count = number of digit positions!

##### JS

for loops, while loops. Build a JS function that generates the full multiplication table up to  $n \times n$ , displayed as an HTML table. Each table cell coloured by the digital root of the product — Vedic maths pattern visualised!

 *Outcome: A coloured multiplication table webpage where colour shows digital root patterns — visually demonstrating Vedic number harmony.*

### WEEK 10

#### JS Functions — Sutras as Reusable Code

##### VEDIC

A sutra is a rule you apply to any valid input. Ekadhikena works for  $1/19$ ,  $1/29$ ,  $1/39$ ... Nikhilam works for any numbers near any base. You don't rewrite the rule each time — you apply it. That's exactly what a function does.

##### JS

function keyword, parameters, return. Write functions: `digitalRoot(n)`, `multiplyByEleven(n)`, `squareEndingFive(n)`, `nearBaseMult(a, b, base)`. Call each from the calculator. Students see that every Vedic sutra becomes one clean function.

 *Outcome: Students refactor their Week 8 calculator to use named functions — one per sutra. Code becomes readable, organised, and elegant — just like the sutras themselves.*

### WEEK 11

#### JS Arrays — Working with Digit Sequences

##### VEDIC

Many Vedic techniques work digit by digit: digital root adds each digit, Nikhilam operates per digit, the "casting out 9s" verification collects and sums digits. Digits of a number = a list = an array.

##### JS

Arrays: `[]`, `.push()`, `.length`, `.forEach()`, `.map()`, `.reduce()`. Convert number to array of digits: `"1234".split("").map(Number)`. Build: sum of digits (digital root), Vedic digit manipulation, "Casting out 9s" verifier as a webpage.

 *Outcome: Students build a "Vedic Verifier" — enter any multiplication and the page confirms correctness using casting out 9s, showing each digit step.*

### WEEK 12

#### CSS Animations + JS — Making Vedic Patterns Move

 **VEDIC**

Vedic art: yantras, mandalas, and rangoli all have mathematical patterns — rotational symmetry (multiples of 360°), reflective symmetry. Digit patterns in 9s table (9,18,27,36,45,54,63,72,81,90) form a visual symmetry when charted.

 **CSS + JS**

@keyframes, animation, transform: rotate(). JS: setInterval() to animate. Build a spinning Vedic yantra made of div elements. Each petal's position calculated from a Vedic proportional formula. Dynamic — spins and pulses.

 *Outcome: Students create an animated Vedic yantra in a browser using pure CSS and JS. A beautiful, share-worthy artefact that demonstrates both cultural and technical mastery.*

## Phase 4: Final Project — Teach What You Know

WEEKS 13–16

**WEEK 13**

### Project Planning — Design Your Vedic Teaching Website

 **VEDIC**

Students pick 2–3 Vedic sutras to teach on their website. Write out: Sanskrit name, English translation, worked example in 5 steps, 3 practice problems with answers. This becomes the content for their website.

 **WEB DESIGN**

Wireframe: Home page, Sutras page, Interactive Calculator page, About page. Sketch on paper first. Choose colour scheme. List which JS functions they'll need. Map content to HTML sections. One week, four pages planned.

 *Deliverable: Paper wireframes for all 4 pages, content written out, JS function list drawn up. Teacher signs off before building starts.*

**WEEK 14**

### Project Build — Structure & Style All Pages

 **VEDIC**

Students write the Vedic content (examples, explanations) in their own words. They must be able to solve any problem on their own site using only the sutra — no looking up answers. The teacher quizzes them on their own content.

 **HTML + CSS**

Build all 4 HTML pages. Shared CSS file (one stylesheet for all pages). Navigation links between pages. Flexbox or grid layout. Images/icons sourced freely. All pages styled consistently. No JS yet — just structure and style.

 *By end: All 4 HTML pages exist, are styled, and navigation works. Content is accurate and readable.*

**WEEK 15**

### Project JS — Add All Interactive Elements

 **VEDIC** **JS**

The interactive calculator must show Vedic steps, not just the final answer. Step 1, Step 2, Step 3 — each appears as the user enters input, mirroring how a Vedic scholar would teach a student. Teaching = debugging = verification.

Attach all JS: calculator with step-by-step reveal, quiz section with score tracking, animated yantra or pattern, smooth scrolling, "copy to clipboard" for sharing sutras. addEventListener, DOM manipulation, innerHTML.

 Website is fully functional. A classmate can use it to learn the Vedic sutras without any help from the creator.

## WEEK 16 — DEMO DAY 🎉

### T.H.Y.N.K Demo Day — Websites Go Live!

#### PRESENTATION

3-minute presentation per student: "My website teaches [sutras]. I chose them because... The hardest part was... The most interesting connection I found was... Anyone can learn [sutra] in 5 minutes using my site."

#### LIVE DEMO

Audience (teachers, parents, other students) uses the website live. Must successfully solve a Vedic problem using the student's calculator. Code walkthrough: show one function and explain it. Receive T.H.Y.N.K Certificate + digital badge.

 Websites shared with school. Best projects featured as teaching resources for younger students. Students may optionally publish to GitHub Pages.

## CLASSROOM MATERIAL

# Sample Exercises from the Program

These are the kinds of paired exercises students work through each week — a Vedic maths puzzle solved mentally, then the same logic coded immediately after.

## Exercise A — Grades 4–5 (Scratch)

Week 4:  $\times 11$  Trick  $\rightarrow$  Scratch Calculator**VEDIC PART (PAPER FIRST — 5 MIN)**Solve these mentally using the  $\times 11$  sutra:**34  $\times$  11**  $\rightarrow$  Split 3\_4, middle = 3+4 = 7  $\rightarrow$  **374****62  $\times$  11**  $\rightarrow$  Split 6\_2, middle = 6+2 = 8  $\rightarrow$  **682****85  $\times$  11**  $\rightarrow$  Split 8\_5, middle = 8+5 = 13, carry  $\rightarrow$  **935****NOW CODE IT IN SCRATCH (30 MIN)**Build a  $\times 11$  calculator that:

- Asks for a 2-digit number
- Uses 3 variables: Tens, Units, Middle
- Sets Middle = Tens + Units
- If Middle < 10: says (Tens)(Middle)(Units)
- If Middle  $\geq$  10: carries the 1 to Tens

**EXTENSION CHALLENGE**

Modify the code to also check if the number IS divisible by 11 (digital root trick). Add a "🎉 Divisible!" message.

🔧 **Starter .sb3 file provided | Teacher answer key included**

**Exercise B — Grades 6–7 (HTML/JS)**Week 10: Functions  $\rightarrow$  Sutra Library**VEDIC PART (PAPER FIRST — 5 MIN)**

Apply each sutra to the number pair (97, 96):

**Nikhilam (base 100):** deficits = 3, 4. Left = 93. Right = 12.  $\rightarrow$  **9312****Digital root check:** Root(9312) = 9+3+1+2 = 15  $\rightarrow$  6. Root(97) $\times$ Root(96) = 7 $\times$ 6 = 42  $\rightarrow$  6. ✓**NOW CODE IT IN JS (30 MIN)**

```
function nikhilam(a, b, base) {
  let dA = base - a;
  let dB = base - b;
  let left  = a - dB;
  let right = dA * dB;
  return left * base + right;
}
```

```
function digitalRoot(n) {
  return n === 0 ? 0
```

```

      : 1 + (n-1) % 9;
    }

```

### EXTENSION CHALLENGE

Write a `verify(a, b)` function that uses `nikhilam()` then checks with `digitalRoot()`. Return `true/false` and display ✓ or ✗ on the page using DOM manipulation.

 **HTML starter template provided | Expected output screenshot included**

## Exercise C — Grades 4–5 (Scratch)

Week 9: Digital Roots → If/Else Divisibility

### VEDIC DIVISIBILITY RULES

**Divisible by 9:** digital root = 9

**Divisible by 3:** digital root = 3, 6, or 9

**Divisible by 2:** last digit even

Test: 738 → 7+3+8 = 18 → 1+8 = 9 → div by 9 AND 3!

### SCRATCH CHALLENGE

Build a "Vedic Divisibility Tester" that:

- Asks for any number
- Calculates its digital root using a variable
- Uses if/else blocks to check all three rules
- Announces all applicable divisors
- Works for any number the student enters

 **Flowchart template provided | Visual step-by-step guide included**

## Exercise D — Grades 6–7 (CSS+JS)

Week 12: Vedic Pattern → Animated Yantra

### VEDIC PATTERN — 9S TABLE CIRCLE

Plot the units digit of each multiple of 9 on a clock-face: 9→9, 18→8, 27→7, 36→6... They go 9,8,7,6,5,4,3,2,1,0 — a perfect symmetrical pattern! Connect the dots → a star polygon. That's a yantra!

### CSS + JS BUILD

```

/* 10 dots on a circle */
.dot {
  position: absolute;
  width: 12px; height: 12px;
  border-radius: 50%;
  background: gold;
  /* JS sets left/top using
    sin/cos of angle */
}
/* Connecting lines drawn
   using SVG in JS */

```

#### ANIMATION LAYER

Add CSS @keyframes rotation so the entire yantra slowly spins. Make each dot pulse. The result is a live, animated Vedic mathematical artwork — generated entirely by student code.



**Starter HTML with SVG setup provided | Video walkthrough for teacher**

#### WHAT SCHOOLS RECEIVE

## The Complete T.H.Y.N.K Teacher Kit

Every school that adopts T.H.Y.N.K receives a complete, ready-to-deliver package. Teachers open the folder and start teaching — no research, no prep from scratch, no extra qualifications needed.



**Teacher Guides (×32 sessions)**



**Student Worksheets (×32 sessions)**

One complete guide per week, for both tracks. Each guide includes:

- ✓ Vedic concept explanation with worked examples (teacher background)
- ✓ Step-by-step lesson plan (5-min Vedic → 30-min code → 10-min share)
- ✓ Common student mistakes and how to address them
- ✓ Extension activities for fast finishers
- ✓ Simplified activity for students who need more support

Print-and-use A4 worksheets, one per session:

- ✓ Vedic warm-up: 5 mental maths problems using the session's sutra
- ✓ Guided coding exercise: step-by-step with blanks to fill in
- ✓ Challenge extension: open-ended creative task
- ✓ Reflection: "I found \_\_ similar between the sutra and the code because \_\_"
- ✓ Take-home: one Vedic trick to teach a family member



### Scratch Project Library

Downloadable .sb3 files for every session:

- ✓ Starter files with sprites, variables, and partial code
- ✓ Complete solution files (teacher reference only)
- ✓ 8 sample student projects to show as inspiration
- ✓ Demo Day project template
- ✓ Hosted on shared Scratch studio — no file management needed



### Web Coding Starter Pack

For Grade 6–7 HTML/CSS/JS sessions:

- ✓ Pre-built HTML templates for each week (students modify, not create blank)
- ✓ CSS stylesheet with Vedic-themed colour variables pre-set
- ✓ JS function stubs with helpful comments
- ✓ Final project 4-page website skeleton
- ✓ VS Code setup guide (works on any Windows computer)



### Assessment & Tracking

Simple, teacher-friendly evaluation tools:

- ✓ Weekly 3-point checklist (Vedic understood / Code works / Connected)



### Teacher Training (1 Day)

Delivered at your school before the program starts:

both)

- ✓ Mini-project rubric (Week 8) with scoring guide
- ✓ Final project rubric with parent-friendly language
- ✓ Progress tracker spreadsheet (one row per student)
- ✓ Certificate template (customisable with student name and sutra mastered)

- ✓ Morning: All 7 Vedic sutras covered, hands-on practice
- ✓ Afternoon: Walk through 3 complete lesson plans end to end
- ✓ Scratch and VS Code setup on school computers
- ✓ Access to WhatsApp teacher support group
- ✓ Quarterly catch-up call with T.H.Y.N.K academic team

## WHAT SCHOOLS NEED TO PROVIDE

# Simple Requirements

T.H.Y.N.K is designed to work with existing school infrastructure. No new hardware, no new software licences, no specialist teachers required.



### Computer Lab

Any existing Windows/Mac lab. Minimum 1 computer per 2 students. Chrome browser installed. Internet for Scratch (or local offline Scratch Desktop). No new purchases needed.



### One Teacher

Your existing computer teacher. No prior Vedic Maths knowledge needed. Comfortable with basic computer use. We train them — one full day before the program starts.



### 45 Minutes Weekly

One session per week. Works on Saturdays, after school, or as a dedicated free period.  
Minimum 12 weeks, recommended 16 weeks for complete curriculum.



### 15–30 Students

Minimum viable group: 15 students per batch.  
Maximum recommended: 30 (1 teacher).  
Multiple batches possible. Mixed-ability welcome — Vedic maths differentiates naturally.

## Ready to Bring T.H.Y.N.K to Your School?

We are currently onboarding schools for the 2025 cohort. The program is designed to run immediately — if you can start in 4 weeks, we can have your teacher trained and materials delivered in time.

 [Express Interest](#)

 [View Full Curriculum](#)

T.H.Y.N.K — The Hub for Young Next-gen Knowledge • Schools • Aligned with NEP 2020 •  
No additional budget required from schools

# T.H.Y.N.K

The Hub for Young Next-gen Knowledge  
Ancient Wisdom. Modern Code. Schools.

This proposal document is confidential and prepared for school leadership review.